FastAPI Learning Roadmap: Building TaskFlow

Comprehensive 2-Week Learning Plan with Daily Tasks

Table of Contents

1. Overview
2. Detailed Phase Explanations
3. 2-Week Daily Breakdown
4. Learning Resources
5. Success Criteria

Overview

This is a complete 14-day learning roadmap designed to take you from absolute beginner to building a **production-ready FastAPI application**. You'll build TaskFlow, a full-stack todo application with user authentication, database persistence, and React frontend integration.

What You'll Learn:

- FastAPI fundamentals and HTTP concepts
- Pydantic data validation
- PostgreSQL database design
- Full CRUD operations
- JWT authentication and authorization
- API architecture and best practices
- Testing and documentation
- Deployment strategies

Time Commitment: 35-40 hours total (3-4 hours daily)

Detailed Phase Explanations

Phase 1: The Basics (Your First API)

Duration: 2-3 days | **Why:** Foundation

FastAPI is a modern Python web framework built on Starlette (for HTTP handling) and Pydantic (for data validation). Understanding the basics means:

- **Request/Response Cycle** - How HTTP requests flow through your application and how responses are sent back
- **Uvicorn** - An ASGI (Asynchronous Server Gateway Interface) server that runs your FastAPI application
- **Decorators** - FastAPI uses `@app.get()`, `@app.post()` to define endpoints

- **Path Parameters** - Variables in the URL (e.g., `/tasks/{task_id}` where `task_id` is extracted from the URL)
- **Query Parameters** - Optional filters in the URL (e.g., `/tasks/?skip=0&limit=10`)

Key Request/Response Flow:

```
Request: GET /tasks/5?skip=0&limit=10
↓
FastAPI routes to the correct handler function
↓
Path Parameter extracted: task_id = 5
Query Parameters extracted: skip = 0, limit = 10
↓
Function executes
↓
Response: {"id": 5, "title": "Learn FastAPI", ...}
```

Phase 2: Data Validation with Pydantic

Duration: 2-3 days | **Why:** Data Integrity

Pydantic is a data validation library that uses Python type hints. It ensures:

- **Type Safety** - Only the correct data types are accepted
- **Automatic Validation** - Pydantic does validation automatically
- **Auto Documentation** - FastAPI generates OpenAPI (Swagger) docs from your models
- **Error Messages** - Clear feedback when data is invalid

Real-World Example:

Without Pydantic (UNSAFE)

```
@app.post("/tasks/")
def create_task(title, description):
# What if someone sends a number for title?
return {"title": title}
```

With Pydantic (SAFE)

```
class TaskCreate(BaseModel):
title: str # MUST be a string
description: str
due_date: Optional[datetime] = None

@app.post("/tasks/")
def create_task(task: TaskCreate):
# FastAPI validates automatically
return task
```

Why Separate Models?

- **TaskCreate** - What the user sends when creating a task (no id or `created_at`)

- **TaskResponse** - What the API returns (includes `id`, `created_at`, `updated_at`)

Phase 3: Database Integration (SQLModel + PostgreSQL)

Duration: 2-3 days | **Why:** Persistence

Data in memory is lost on server restart. A database persists data permanently.

Technology Stack:

- **PostgreSQL** - Powerful, reliable relational database
- **SQLModel** - Combines SQLAlchemy (database ORM) + Pydantic (validation)

Database Flow:

FastAPI Request
↓
Pydantic validates data
↓
SQLModel converts to database record
↓
INSERT into PostgreSQL
↓
Database returns new record with ID
↓
FastAPI returns JSON response

Phase 4: CRUD Operations (The Core Logic)

Duration: 2-3 days | **Why:** Core Functionality

CRUD = Create, Read, Update, Delete. This is where your API does actual work.

Operation	HTTP Method	URL Example	Purpose
Create	POST	/tasks/	Insert new record
Read	GET	/tasks/ or /tasks/{id}	Retrieve data
Update	PUT/PATCH	/tasks/{id}	Modify existing record
Delete	DELETE	/tasks/{id}	Remove record

Phase 5: Structure and Architecture

Duration: 1-2 days | **Why:** Maintainability

As your API grows, organizing code becomes critical.

Layered Monolith Architecture:

```
main.py
├── app/
│  ├── api/
│  │  └── endpoints/
│  │  └── tasks.py (APIRouter)
│  ├── models/
│  │  └── task.py (Database models)
│  ├── schemas/
│  │  └── task.py (Pydantic models)
│  ├── services/
│  │  └── task_service.py (Business logic)
│  ├── database.py (Configuration)
│  └── dependencies.py (Shared dependencies)
```

Phase 6: Authentication and Security

Duration: 2-3 days | **Why:** Protect Your App

Password Security:

- Never store plain passwords
- Use bcrypt to hash passwords (one-way encryption)
- Compare hashes during login

JWT (JSON Web Tokens):

- After successful login, generate a token
- Token is signed with a secret key
- Client sends token with each request: `Authorization: Bearer <token>`
- Server verifies token signature

Phase 7: Connecting to React

Duration: 1-2 days | **Why:** Full Integration

CORS (Cross-Origin Resource Sharing):

- Browser prevents cross-origin requests by default
- Configure FastAPI to allow requests from React frontend
- Simple one-time setup in main.py

2-Week Daily Breakdown

WEEK 1: Core FastAPI and Database Fundamentals

Day 1: Project Setup and Hello World (Phase 1.1)

Time Commitment: 3-4 hours | **Status:** Foundation

Morning Session (2 hours):

1. Create project directory
2. Set up Python virtual environment
3. Install FastAPI and Uvicorn
4. Create `main.py` with basic endpoints
5. Start server with `uvicorn main:app --reload`
6. Test endpoints at http://localhost:8000
7. Explore Swagger UI at http://localhost:8000/docs

Afternoon Session (1-2 hours):

8. Experiment with different return types
9. Read FastAPI First Steps documentation
10. **Checkpoint:** Your server runs and returns JSON responses ✓

Deliverables:

- Working FastAPI server on localhost:8000
- Understanding of request/response cycle
- Familiarity with Swagger UI

Day 2: Path and Query Parameters (Phase 1.2)

Time Commitment: 3-4 hours | **Status:** Building Endpoints

Morning Session (2 hours):

1. Implement path parameters: `/tasks/{task_id}`
2. Experiment with different parameter types
3. Implement query parameters: `?skip=0&limit=10`
4. Combine path and query parameters

Afternoon Session (1-2 hours):

5. Test URL combinations in browser and Swagger
6. Understand parameter validation
7. Learn about optional parameters

Deliverables:

- Path parameter endpoints
- Query parameter endpoints
- Parameter validation

Day 3: Pydantic Schemas (Phase 2.1 and 2.2)

Time Commitment: 4-5 hours | **Status:** Data Validation

Morning Session (2 hours):

1. Create `schemas.py` with Pydantic models
2. Define TaskCreate and TaskResponse schemas
3. Add field constraints using Pydantic Field
4. Test request body validation

Afternoon Session (2-3 hours):

5. Create endpoints that use schemas
6. Understand response_model
7. Implement proper error handling

Deliverables:

- Pydantic schemas for requests/responses
- Request body validation
- Swagger documentation of schemas

Day 4: Database Setup (Phase 3.1 and 3.2)

Time Commitment: 4-5 hours | **Status:** Persistence Layer

Morning Session (2-3 hours):

1. Install SQLModel and database packages
2. Set up PostgreSQL (Docker or local)
3. Create `database.py` with SQLModel configuration
4. Create `models.py` with Task model
5. Initialize database on startup

Afternoon Session (2 hours):

6. Test database connection
7. Verify table creation
8. Understand database relationships

Deliverables:

- Working PostgreSQL database
- Database engine and session management
- Task model and tables

Day 5: CRUD Operations Part 1 - Create and Read (Phase 4.1 and 4.2)

Time Commitment: 4-5 hours | **Status:** Core API Logic

Morning Session (2-3 hours):

1. Create `endpoints/tasks.py` APIRouter
2. Implement POST /tasks/ (create)

3. Implement GET /tasks/ (list)
4. Implement GET /tasks/{id} (get one)

Afternoon Session (2 hours):

5. Test all endpoints in Swagger
6. Test pagination
7. Verify data persists

Deliverables:

- POST /tasks/ endpoint
- GET /tasks/ endpoint with pagination
- GET /tasks/{id} endpoint
- Proper HTTP status codes

Day 6: CRUD Operations Part 2 - Update and Delete (Phase 4.3 and 4.4)

Time Commitment: 4-5 hours | **Status:** Complete CRUD

Morning Session (2-3 hours):

1. Create TaskUpdate schema
2. Implement PUT /tasks/{id} (full update)
3. Implement PATCH /tasks/{id} (partial update)
4. Implement DELETE /tasks/{id}

Afternoon Session (2 hours):

5. Test all operations in Swagger
6. Test edge cases
7. Verify timestamps update

Deliverables:

- PUT and PATCH endpoints
- DELETE endpoint
- Proper error handling (404)
- Timestamp management

Day 7: Architecture Refactoring (Phase 5)

Time Commitment: 4-5 hours | **Status:** Code Organization

Morning Session (2-3 hours):

1. Create layered project structure
2. Create services/task_service.py with business logic
3. Create dependencies.py for dependency injection
4. Refactor endpoints to use services

Afternoon Session (2 hours):

5. Verify all endpoints still work
6. Test service layer

7. Understand separation of concerns

Deliverables:

- Reorganized file structure
- Service layer implementation
- Dependency injection pattern
- Clean architecture

WEEK 2: Security, Testing and Integration

Day 8: Authentication - User Model and Registration (Phase 6.1)

Time Commitment: 4-5 hours | **Status:** User System

Morning Session (2-3 hours):

1. Install authentication packages
2. Create User model in `models.py`
3. Create password hashing in `core/security.py`
4. Create auth schemas
5. Create `services/auth_service.py`

Afternoon Session (2 hours):

6. Create auth endpoints in `endpoints/auth.py`
7. Test user registration
8. Verify password hashing

Deliverables:

- User model
- Password hashing with bcrypt
- Registration endpoint
- Duplicate email validation

Day 9: JWT Authentication and Protected Endpoints (Phase 6.2 and 6.3)

Time Commitment: 4-5 hours | **Status:** Login and Authorization

Morning Session (2-3 hours):

1. Implement login endpoint
2. Create JWT token generation
3. Create `get_current_user` dependency
4. Test JWT token creation

Afternoon Session (2 hours):

5. Protect task endpoints with authentication
6. Test protected endpoints
7. Verify user isolation

Deliverables:

- Login endpoint
- JWT token generation
- Authentication dependency
- Protected endpoints
- User isolation

Day 10: CORS Setup and API Refinement (Phase 7.1)

Time Commitment: 3-4 hours | **Status:** Frontend Ready

Morning Session (2 hours):

1. Configure CORS middleware
2. Allow frontend origins
3. Add error handling middleware
4. Add request logging

Afternoon Session (1-2 hours):

5. Add API versioning
6. Add health check endpoint
7. Test CORS configuration

Deliverables:

- CORS configuration
- Error handling
- Request logging
- Health check endpoint
- API versioning

Day 11: Testing and Error Handling

Time Commitment: 4-5 hours | **Status:** Reliability

Morning Session (2-3 hours):

1. Install testing frameworks (pytest)
2. Create test directory structure
3. Create test configuration
4. Write authentication tests
5. Write task operation tests

Afternoon Session (2 hours):

6. Run test suite
7. Check test coverage
8. Add error handling

Deliverables:

- Comprehensive test suite
- Auth tests
- Task operation tests

- 80% coverage

Day 12: Documentation and Deployment Prep

Time Commitment: 4-5 hours | **Status:** Production Ready

Morning Session (2-3 hours):

1. Write comprehensive README.md
2. Create requirements.txt
3. Create .env template
4. Create environment configuration module
5. Set up .gitignore

Afternoon Session (2 hours):

6. Add Swagger metadata
7. Create Docker configuration
8. Create docker-compose.yml
9. Create deployment checklist

Deliverables:

- README documentation
- Dependencies file
- Environment configuration
- Docker setup
- Deployment checklist

Day 13: Frontend Integration Testing

Time Commitment: 4-5 hours | **Status:** Full Stack

Morning Session (2-3 hours):

1. Create React project
2. Install Axios
3. Create API client service
4. Create React components
5. Implement authentication flow

Afternoon Session (2 hours):

6. Test full registration flow
7. Test CRUD operations
8. Test error handling
9. Verify end-to-end functionality

Deliverables:

- React frontend connected
- Authentication flow
- Full CRUD from frontend
- Error handling

- End-to-end tests

Day 14: Final Review and Optimization

Time Commitment: 4-5 hours | **Status:** Production Launch

Morning Session (2-3 hours):

1. Code review and cleanup
2. Add comprehensive docstrings
3. Optimize performance
4. Harden security
5. Add logging

Afternoon Session (2 hours):

6. Final testing
7. Create deployment guide
8. Document next steps

Deliverables:

- Clean, documented code
- Performance optimized
- Security hardened
- Deployment guide
- Production-ready API

Learning Resources

Official Documentation

- FastAPI Official Tutorial
- Pydantic Documentation
- SQLModel Documentation
- PostgreSQL Documentation

Recommended Learning Materials

- DeepLearning.AI FastAPI Course
- Real Python FastAPI Articles
- Starlette Documentation
- SQLAlchemy Documentation

Tools

- Swagger UI (`/docs`)
- ReDoc (/redoc)
- Postman
- pgAdmin
- DBeaver

Community

- Stack Overflow FastAPI tag
- GitHub Discussions
- Discord FastAPI server
- Reddit r/FastAPI

Success Criteria

By End of Week 1

- ✅ API runs locally
- ✅ 6+ endpoints working
- ✅ Pydantic validation active
- ✅ PostgreSQL database connected
- ✅ Data persistence working
- ✅ Code organized in layers

By End of Week 2

- ✅ Authentication implemented
- ✅ All endpoints protected
- ✅ Users isolated
- ✅ CORS configured
- ✅ 80%+ test coverage
- ✅ Full Swagger documentation
- ✅ Frontend integrated
- ✅ Deployment-ready

Knowledge Checkpoints

- Understand HTTP request/response cycle
- Explain path vs query parameters
- Implement request validation
- Design database schema
- Build CRUD operations
- Implement authentication
- Organize code architecture
- Write tests
- Document API

Time Management Tips

Daily Workflow

1. **Morning (2-3 hours):** Learning and implementation
2. **Afternoon (1-2 hours):** Testing and refinement
3. **Evening (optional):** Review and prepare next day

Staying on Track

- Take daily notes
- Commit code daily with Git
- Test immediately after coding
- Ask questions in communities
- Review previous day's content
- Don't skip foundations

If Behind Schedule

- Phase 1-2: Don't rush, foundational
- Phase 3-4: Can be condensed
- Phase 5-7: Focus on minimum viable
- Testing: Focus on critical paths

Final Notes

This roadmap takes you from beginner to **production-ready API developer**. Each phase builds on the previous one. Don't skip steps, understand concepts deeply, and build incrementally.

By Day 14, you will have:

- ✨ A working API in production
- 🧠 Deep FastAPI understanding
- 🔐 Real authentication system
- 🗄️ Database design knowledge
- ✅ Testing and quality practices
- 🚀 Deployment knowledge

The key to success is consistent, focused practice. Good luck! 🍀